Tarayıcıdan ilk randevuya

Başlat.cmd dosyasını açın (dosya adı Baslat.cmd). Tarayıcıda http://localhost:8080 adresine gidin. Terminal açık kaldığı sürece uygulama çalışır. Kayıtlar veri klasöründe saklanır.

1. İlk yönetici hesabını oluşturun, ardından aynı bilgilerle giriş yapın. Merkez şubesini 09:00–17:00 saatleriyle ekleyin. Bu şubeye Dahiliye bölümü ve 30 dakikalık bir muayene tanımlayın.

2. Aynı bölüm için bir doktor kaydedin. Çalışana hizmet ata formunda doktor ile muayeneyi eşleştirin. Yarın için 09:00–10:00 çalışma aralığı ekleyin.

3. Çıkış yapın. Hasta kaydı formunu doldurun ve giriş yapın. Randevu ara bölümünden çalışanın hizmetini ve yarının tarihini seçin. 09:00 seçeneğiyle talep oluşturun.

4. Doktor hesabıyla giriş yapıp talebi onaylayın. Hasta hesabına geri döndüğünüzde randevu tablosunda güncellenmiş durumu görebilirsiniz.

İstek akışı. Tarayıcı bir form gönderir. Spring MVC ilgili metodu çağırır. Metot oturumu denetler, JDBC ile H2 üzerinden kayıt yapar ve kullanıcıyı panele yönlendirir. Her tarayıcı oturumunun hesap bilgileri ayrıdır.

src/main/java/tr/ndp/hastane/Hastane.java

Hastane / 1–11

SpringApplication.run Spring bileşenlerini yükler ve gömülü Tomcat sunucusunu başlatır.

```
1    package tr.ndp.hastane;
2
3    import org.springframework.boot.SpringApplication;
4    import org.springframework.boot.autoconfigure.SpringBootApplication;
5
6    @SpringBootApplication
7    public class Hastane {
8        public static void main(String[] args) {
9            SpringApplication.run(Hastane.class, args);
10       }
11   }
```

Bolme / 1–7

Her panel parçası aynı goster metodunu sunar. AnaSayfa çağırdığında her sınıf kendi rolüne uygun formu döndürür. Bu davranış çok biçimliliğe örnektir.

```
1    package tr.ndp.hastane;
2
3    import jakarta.servlet.http.HttpSession;
4
5    public interface Bolme {
6        String goster(HttpSession oturum);
7    }
```

AnaSayfa / 1–18

GET / ana sayfadır. Oturuma bir form anahtarı atanır. Spring tarafından sağlanan panel bileşenleri sırayla HTML üretir.

```
1    package tr.ndp.hastane;
2
3    import jakarta.servlet.http.HttpSession;
4    import java.util.List;
5    import java.util.UUID;
6    import org.springframework.web.bind.annotation.*;
7
8    @RestController
9    public record AnaSayfa(List<Bolme> bolmeler) {
10       @GetMapping(value="/",produces="text/html;charset=UTF-8")
11       public String ana(HttpSession s) {
12           if(s.getAttribute("anahtar")==null) s.setAttribute("anahtar",UUID.randomUUID().toString());
13           StringBuilder html=new StringBuilder();
14           if(s.getAttribute("ad")!=null) html.append("<p
14          class='kimlik'>").append(Sayfa.metin(s.getAttribute("ad"))).append(" ·
14          ").append(Sayfa.metin(s.getAttribute("rol"))).append("</p>");
15           for(Bolme bolme:bolmeler) html.append(bolme.goster(s));
16           return Sayfa.cerceve(html.toString());
17       }
18   }
```

Sayfa / 1–30

Ortak form, alan, tablo ve HTML iskeleti burada üretilir. metin kullanıcı girdisini HTML içinde güvenli gösterir. yetki oturum rolünü doğrular.

```
1    package tr.ndp.hastane;
2
3    import jakarta.servlet.http.HttpSession;
4    import java.util.List;
5    import java.util.Map;
6    import org.springframework.web.util.HtmlUtils;
7
8    public final class Sayfa {
9        public static String metin(Object s) {
10           return HtmlUtils.htmlEscape(String.valueOf(s));
11       }
12
13       public static boolean rol(HttpSession s, String rol) {
14           return rol.equals(s.getAttribute("rol"));
15       }
16
17       public static long yetki(HttpSession s, String... roller) {
18           for (String rol : roller) if (rol(s,rol)) return (Long)s.getAttribute("id");
19           throw new IllegalArgumentException("Bu işlem için uygun hesapla giriş yapın.");
20       }
21
22       public static String alan(String ad,String etiket,String tur) {
23           String ek = tur.equals("number") ? " min='0' step='any'" : "";
24           return "<label>"+etiket+"<input required name='"+ad+"' type='"+tur+"'"+ek+"></label>";
25       }
26
27       public static String sec(String ad,String etiket,List<Map<String,Object>> satirlar) {
28           StringBuilder html=new StringBuilder("<label>"+etiket+"<select required name='"+ad+"'><option
28          value=''>Seçiniz</option>");
29           for(var r:satirlar) html.append("<option
29          value='").append(metin(r.get("ID"))).append("'>").append(metin(r.get("ad"))).append("</option>");
30           return html+"</select></label>";
```

Sayfa / 31–54

Ortak form, alan, tablo ve HTML iskeleti burada üretilir. metin kullanıcı girdisini HTML içinde güvenli gösterir. yetki oturum rolünü doğrular.

```
31      }
32
33      public static String form(HttpSession s,String yol,String baslik,String alanlar,String dugme) {
34          return "<section><h2>"+baslik+"</h2><form method='post' action='"+yol+"'><input type='hidden' name='anahtar'
34          value='"
35                  +s.getAttribute("anahtar")+"'>"+alanlar+"<button>"+dugme+"</button></form></section>";
36      }
37
38      public static String tablo(List<Map<String,Object>> sonuc) {
39          if(sonuc.isEmpty()) return "<p>Henüz kayıt yok.</p>";
40          StringBuilder t=new StringBuilder("<div class='kaydir'><table><thead><tr>");
41          sonuc.get(0).keySet().forEach(k -> t.append("<th>").append(metin(k)).append("</th>"));
42          t.append("</tr></thead><tbody>");
43          for(var r:sonuc) {
44              t.append("<tr>");
45              r.values().forEach(v -> t.append("<td>").append(metin(v)).append("</td>"));
46              t.append("</tr>");
47          }
48          return t+"</tbody></table></div>";
49      }
50
51      public static String cerceve(String icerik) {
52          return "<!doctype html><html lang='tr'><meta charset='utf-8'><meta name='viewport' content='width=device-
52         width,initial-scale=1'><title>Klinik | Randevu sistemi</title><link rel='stylesheet'
52         href='/stil.css'><main><header><small>KLİNİK  /  YEREL HASTANE SİSTEMİ</small><h1>Randevu
52         merkezi</h1></header>"+icerik+"</main></html>";
53      }
54  }
```

Koruma / 1–25

Her POST isteğinde tarayıcıdan gelen anahtar oturumdaki değerle karşılaştırılır. Uyuşmazsa form işlemi çalıştırılmadan 403 döner.

```
1   package tr.ndp.hastane;
2
3   import jakarta.servlet.*;
4   import jakarta.servlet.http.*;
5   import java.io.IOException;
6   import org.springframework.stereotype.Component;
7   import org.springframework.web.filter.OncePerRequestFilter;
8
9   @Component
10  public class Koruma extends OncePerRequestFilter {
11      protected void doFilterInternal(HttpServletRequest req,HttpServletResponse res,FilterChain zincir)
12              throws ServletException,IOException {
13          res.setHeader("Cache-Control","no-store");
14          res.setHeader("X-Frame-Options","DENY");
15          if(req.getMethod().equals("POST")) {
16              var s=req.getSession(false);
17              String token=req.getParameter("anahtar");
18              if(s==null || token==null || !token.equals(s.getAttribute("anahtar"))) {
19                  res.sendError(403);
20                  return;
21              }
22          }
23          zincir.doFilter(req,res);
24      }
25  }
```

src/main/java/tr/ndp/hastane/Hatalar.java

Hatalar / 1–15

Geçersiz seçimler ve veri kısıtı hataları, kullanıcının panele dönebileceği bir hata sayfasına çevrilir.

```
1    package tr.ndp.hastane;
2
3    import org.springframework.dao.DataAccessException;
4    import org.springframework.http.*;
5    import org.springframework.web.bind.annotation.*;
6
7    @RestControllerAdvice
8    public class Hatalar {
9        @ExceptionHandler({IllegalArgumentException.class,DataAccessException.class,java.time.DateTimeException.class,Ar
9           ithmeticException.class})
10       ResponseEntity<String> hata(RuntimeException e) {
11           String mesaj=e instanceof DataAccessException ? "Kayıt yapılamadı. Seçimleri, saatleri ve yinelenen
11          kayıtları kontrol edin." : e.getMessage();
12           return ResponseEntity.badRequest().contentType(MediaType.valueOf("text/html;charset=UTF-8"))
13                   .body(Sayfa.cerceve("<section><h2>İşlem tamamlanamadı</h2><p>"+Sayfa.metin(mesaj)+"</p><a
13          href='/'>Panele dön</a></section>"));
14       }
15   }
```

schema / 1–32

Tablolar kimliklerle bağlanır. Süre, ücret ve saat sırası kayıt sırasında kontrol edilir. Randevu bitişi ve ücreti talep anındaki değerlerle saklanır.

```
1    CREATE TABLE IF NOT EXISTS kilit (id INT PRIMARY KEY);
2    MERGE INTO kilit KEY(id) VALUES(1);
3
4    CREATE TABLE IF NOT EXISTS kurum (
5        id BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
6        ad VARCHAR UNIQUE NOT NULL, ac TIME NOT NULL, kapa TIME NOT NULL, CHECK (ac < kapa)
7    );
8    CREATE TABLE IF NOT EXISTS bolum (
9        id BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
10       kurum BIGINT REFERENCES kurum NOT NULL, ad VARCHAR NOT NULL, UNIQUE(kurum, ad)
11   );
12   CREATE TABLE IF NOT EXISTS hizmet (
13       id BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
14       bolum BIGINT REFERENCES bolum NOT NULL, ad VARCHAR NOT NULL,
15       sure INT CHECK(sure BETWEEN 1 AND 1440) NOT NULL,
16       ucret DECIMAL(12,2) CHECK(ucret >= 0) NOT NULL, UNIQUE(bolum, ad)
17   );
18   CREATE TABLE IF NOT EXISTS kisi (
19       id BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
20       hesap VARCHAR UNIQUE NOT NULL, ad VARCHAR NOT NULL, parola VARCHAR NOT NULL,
21       rol VARCHAR NOT NULL, bolum BIGINT REFERENCES bolum, uzmanlik VARCHAR
22   );
23   CREATE TABLE IF NOT EXISTS sunum (
24       kisi BIGINT REFERENCES kisi, hizmet BIGINT REFERENCES hizmet, PRIMARY KEY(kisi,hizmet)
25   );
26   CREATE TABLE IF NOT EXISTS mesai (
27       kisi BIGINT REFERENCES kisi, gun DATE NOT NULL, ilk TIME NOT NULL, son TIME NOT NULL,
28       CHECK(ilk < son), PRIMARY KEY(kisi,gun,ilk,son)
29   );
30   CREATE TABLE IF NOT EXISTS randevu (
31       id BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
32       hasta BIGINT REFERENCES kisi NOT NULL, personel BIGINT REFERENCES kisi NOT NULL,
```

schema / 33–35

Tablolar kimliklerle bağlanır. Süre, ücret ve saat sırası kayıt sırasında kontrol edilir. Randevu bitişi ve ücreti talep anındaki değerlerle saklanır.

```
33      hizmet BIGINT REFERENCES hizmet NOT NULL, gun DATE NOT NULL,
34      ilk TIME NOT NULL, son TIME NOT NULL, ucret DECIMAL(12,2) NOT NULL, durum VARCHAR NOT NULL
35  );
```

HesapServisi / 1–28

Hesap açarken parolanın BCrypt özeti saklanır. İlk yönetici oluşturulurken satır kilidi ikinci bir ilk kayıt yarışını önler. gir yalnızca doğrulanmış hesabı döndürür.

```
1    package tr.ndp.hastane;
2
3    import java.nio.charset.StandardCharsets;
4    import java.util.Locale;
5    import org.springframework.jdbc.core.JdbcTemplate;
6    import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
7    import org.springframework.stereotype.Service;
8    import org.springframework.transaction.annotation.Transactional;
9
10   @Service
11   public class HesapServisi {
12       private final JdbcTemplate db;
13       private final BCryptPasswordEncoder sifre=new BCryptPasswordEncoder();
14       public HesapServisi(JdbcTemplate db) { this.db=db; }
15
16       public boolean kurulum() {
17           return db.queryForObject("select count(*) from kisi where rol='Yönetici'",Integer.class)==0;
18       }
19
20       @Transactional
21       public void ekle(String ad,String hesap,String parola,String rol,Long bolum,String uzmanlik) {
22           if(ad.isBlank() || hesap.isBlank() || parola.length()<8 ||
22          parola.getBytes(StandardCharsets.UTF_8).length>72)
23               throw new IllegalArgumentException("Ad ve kullanıcı adı girin. Parola en az 8 karakter ve en fazla 72
23          bayt olmalı.");
24           if(rol.equals("Yönetici")) {
25               db.queryForObject("select id from kilit where id=1 for update",Integer.class);
26               if(!kurulum()) throw new IllegalArgumentException("İlk kurulum tamamlanmış.");
27           }
28           db.update("insert into kisi(ad,hesap,parola,rol,bolum,uzmanlik)
28          values(?,?,?,?,?,?)",ad.strip(),hesap.strip().toLowerCase(Locale.ROOT),sifre.encode(parola),rol,bolum,uzmanli
28          k);
```

HesapServisi / 29–38

Hesap açarken parolanın BCrypt özeti saklanır. İlk yönetici oluşturulurken satır kilidi ikinci bir ilk kayıt yarışını önler. gir yalnızca doğrulanmış hesabı döndürür.

```
29      }
30
31      public record Kimlik(long id,String ad,String rol) {}
32      public Kimlik gir(String hesap,String parola) {
33          var sonuc=db.queryForList("select id,ad,rol,parola from kisi where
33         hesap=?",hesap.strip().toLowerCase(Locale.ROOT));
34          if(sonuc.isEmpty() || !sifre.matches(parola,(String)sonuc.get(0).get("parola"))) throw new
34         IllegalArgumentException("Kullanıcı adı veya parola hatalı.");
35          var r=sonuc.get(0);
36          return new Kimlik(((Number)r.get("ID")).longValue(),(String)r.get("ad"),(String)r.get("rol"));
37      }
38  }
```

Kimlik / 1–30

Giriş, kayıt ve çıkış formları ayrı adreslere gönderilir. Girişten sonra oturum kimliği yenilenir.
Çıkışta oturum geçersiz kılınır.

```
1    package tr.ndp.hastane;
2
3    import jakarta.servlet.http.*;
4    import org.springframework.core.annotation.Order;
5    import org.springframework.stereotype.Controller;
6    import org.springframework.web.bind.annotation.*;
7    import static tr.ndp.hastane.Sayfa.*;
8
9    @Controller
10   @Order(1)
11   public record Kimlik(HesapServisi hesaplar) implements Bolme {
12       static String alanlar() {
13           return alan("ad","Ad soyad","text")+alan("hesap","Kullanıcı adı","text")+alan("parola","Parola (en az 8
13          karakter)","password");
14       }
15
16       public String goster(HttpSession s) {
17           if(s.getAttribute("id")!=null) return form(s,"/cikis","Oturum","","Çıkış yap");
18           if(hesaplar.kurulum()) return form(s,"/kurulum","İlk yönetici hesabı",alanlar(),"Hesap oluştur");
19           return form(s,"/giris","Oturum aç",alan("hesap","Kullanıcı
19          adı","text")+alan("parola","Parola","password"),"Giriş yap")
20                   +form(s,"/hasta","Hasta kaydı",alanlar(),"Hasta hesabı oluştur");
21       }
22
23       @PostMapping("/kurulum")
24       String kurulum(@RequestParam String ad,@RequestParam String hesap,@RequestParam String parola) {
25           hesaplar.ekle(ad,hesap,parola,"Yönetici",null,null);
26           return "redirect:/";
27       }
28
29       @PostMapping("/hasta")
30       String hasta(@RequestParam String ad,@RequestParam String hesap,@RequestParam String parola) {
```

Kimlik / 31–48

Giriş, kayıt ve çıkış formları ayrı adreslere gönderilir. Girişten sonra oturum kimliği yenilenir. Çıkışta oturum geçersiz kılınır.

```
31          hesaplar.ekle(ad,hesap,parola,"Hasta",null,null);
32          return "redirect:/";
33      }
34
35      @PostMapping("/giris")
36      String gir(HttpServletRequest req,@RequestParam String hesap,@RequestParam String parola) {
37          var k=hesaplar.gir(hesap,parola);
38          req.changeSessionId();
39          var s=req.getSession();
40          s.setAttribute("id",k.id());
41          s.setAttribute("ad",k.ad());
42          s.setAttribute("rol",k.rol());
43          return "redirect:/";
44      }
45
46      @PostMapping("/cikis")
47      String cik(HttpSession s) { s.invalidate(); return "redirect:/"; }
48  }
```

Katalog / 1–13

Kimlikler şube, bölüm veya çalışan adlarıyla eşleştirilir. Formdaki okunabilir seçeneğin değeri veritabanı kimliğidir.

```
1    package tr.ndp.hastane;
2
3    import org.springframework.jdbc.core.JdbcTemplate;
4    import org.springframework.stereotype.Component;
5    import static tr.ndp.hastane.Sayfa.sec;
6
7    @Component
8    public record Katalog(JdbcTemplate db) {
9        public String kurum() { return sec("kurum","Şube",db.queryForList("select id,ad from kurum order by ad")); }
10       public String bolum() { return sec("bolum","Bölüm",db.queryForList("select b.id,k.ad || ' / ' || b.ad ad from
10          bolum b join kurum k on k.id=b.kurum order by ad")); }
11       public String hizmet() { return sec("hizmet","Hizmet",db.queryForList("select h.id,k.ad || ' / ' || b.ad || ' /
11          ' || h.ad ad from hizmet h join bolum b on b.id=h.bolum join kurum k on k.id=b.kurum order by ad")); }
12       public String personel() { return sec("kisi","Çalışan",db.queryForList("select p.id,k.ad || ' / ' || b.ad || ' /
12           ' || p.ad ad from kisi p join bolum b on b.id=p.bolum join kurum k on k.id=b.kurum order by ad")); }
13   }
```

Subeler / 1–31

Yönetici şube açılış ve kapanışını belirler. Bölüm oluşturulurken kurum numarası yabancı anahtar olarak saklanır.

```
1    package tr.ndp.hastane;
2
3    import jakarta.servlet.http.HttpSession;
4    import org.springframework.stereotype.Controller;
5    import org.springframework.core.annotation.Order;
6    import org.springframework.web.bind.annotation.*;
7    import java.time.*;
8    import static tr.ndp.hastane.Sayfa.*;
9
10   @Controller
11   @Order(4)
12   public record Subeler(Katalog k) implements Bolme {
13       public String goster(HttpSession s) {
14           if(!rol(s,"Yönetici")) return "";
15           return form(s,"/kurum","Şube tanımla",alan("ad","Şube
15          adı","text")+alan("ac","Açılış","time")+alan("kapa","Kapanış","time"),"Şube ekle")
16                   +form(s,"/bolum","Bölüm tanımla",k.kurum()+alan("ad","Bölüm adı","text"),"Bölüm ekle");
17       }
18
19       @PostMapping("/kurum")
20       String kurum(HttpSession s,@RequestParam String ad,@RequestParam LocalTime ac,@RequestParam LocalTime kapa) {
21           yetki(s,"Yönetici");
22           k.db().update("insert into kurum(ad,ac,kapa) values(?,?,?)",ad.strip(),ac,kapa);
23           return "redirect:/";
24       }
25
26       @PostMapping("/bolum")
27       String bolum(HttpSession s,@RequestParam long kurum,@RequestParam String ad) {
28           yetki(s,"Yönetici");
29           k.db().update("insert into bolum(kurum,ad) values(?,?)",kurum,ad.strip());
30           return "redirect:/";
31       }
```

Subeler / 32–32

Yönetici şube açılış ve kapanışını belirler. Bölüm oluşturulurken kurum numarası yabancı anahtar olarak saklanır.

```
32  }
```

Hizmetler / 1–24

Hizmet, bölüme bağlanır. Süre dakika cinsindendir. Ücret, iki ondalık basamağa sığmıyorsa kayıt reddedilir.

```
1    package tr.ndp.hastane;
2
3    import jakarta.servlet.http.HttpSession;
4    import org.springframework.stereotype.Controller;
5    import org.springframework.core.annotation.Order;
6    import org.springframework.web.bind.annotation.*;
7    import java.math.*;
8    import static tr.ndp.hastane.Sayfa.*;
9
10   @Controller
11   @Order(5)
12   public record Hizmetler(Katalog k) implements Bolme {
13       public String goster(HttpSession s) {
14           if(!rol(s,"Yönetici")) return "";
15           return form(s,"/hizmet","Hizmet tanımla",k.bolum()+alan("ad","Hizmet adı","text")+alan("sure","Süre
15          (dk)","number")+alan("ucret","Ücret (TL)","number"),"Hizmet ekle");
16       }
17
18       @PostMapping("/hizmet")
19       String hizmet(HttpSession s,@RequestParam long bolum,@RequestParam String ad,@RequestParam int
19          sure,@RequestParam BigDecimal ucret) {
20           yetki(s,"Yönetici");
21           k.db().update("insert into hizmet(bolum,ad,sure,ucret)
21          values(?,?,?,?)",bolum,ad.strip(),sure,ucret.setScale(2,RoundingMode.UNNECESSARY));
22           return "redirect:/";
23       }
24   }
```

Calisanlar / 1–29

Çalışanın rolü ve uzmanlığı kayıt edilir. Hizmet ataması yalnızca çalışanla hizmetin bölümü aynı ise başarılı olur.

```
1    package tr.ndp.hastane;
2
3    import jakarta.servlet.http.HttpSession;
4    import org.springframework.stereotype.Controller;
5    import org.springframework.core.annotation.Order;
6    import org.springframework.web.bind.annotation.*;
7    import java.util.List;
8    import static tr.ndp.hastane.Sayfa.*;
9
10   @Controller
11   @Order(6)
12   public record Calisanlar(Katalog k, HesapServisi hesaplar) implements Bolme {
13       public String goster(HttpSession s) {
14           if(!rol(s,"Yönetici")) return "";
15           return form(s,"/calisan","Çalışan
15          kaydet",Kimlik.alanlar()+k.bolum()+alan("uzmanlik","Uzmanlık","text")+"<label>Rol<select
15          name='tur'><option>Doktor</option><option>Sağlık personeli</option></select></label>","Kaydet")
16                   +form(s,"/ata","Çalışana hizmet ata",k.personel()+k.hizmet(),"Ata");
17       }
18
19       @PostMapping("/calisan")
20       String calisan(HttpSession s,@RequestParam String ad,@RequestParam String hesap,@RequestParam String
20          parola,@RequestParam long bolum,@RequestParam String uzmanlik,@RequestParam String tur) {
21           yetki(s,"Yönetici");
22           if(!List.of("Doktor","Sağlık personeli").contains(tur)) throw new IllegalArgumentException("Rol geçersiz.");
23           hesaplar.ekle(ad,hesap,parola,tur,bolum,uzmanlik);
24           return "redirect:/";
25       }
26
27       @PostMapping("/ata")
28       String ata(HttpSession s,@RequestParam long kisi,@RequestParam long hizmet) {
29           yetki(s,"Yönetici");
```

Calisanlar / 30–34

Çalışanın rolü ve uzmanlığı kayıt edilir. Hizmet ataması yalnızca çalışanla hizmetin bölümü aynı ise başarılı olur.

```
30           int n=k.db().update("insert into sunum select p.id,h.id from kisi p join hizmet h on h.bolum=p.bolum where
30          p.id=? and h.id=?",kisi,hizmet);
31           if(n!=1) throw new IllegalArgumentException("Çalışan ve hizmet aynı bölümde olmalı.");
32           return "redirect:/";
33       }
34   }
```

Mesailer / 1–26

Gün ve zaman aralığı alınır. Şube saatlerinin dışındaki mesai eklenemez.

```
1    package tr.ndp.hastane;
2
3    import jakarta.servlet.http.HttpSession;
4    import org.springframework.stereotype.Controller;
5    import org.springframework.core.annotation.Order;
6    import org.springframework.web.bind.annotation.*;
7    import java.time.*;
8    import static tr.ndp.hastane.Sayfa.*;
9
10   @Controller
11   @Order(7)
12   public record Mesailer(Katalog k) implements Bolme {
13       public String goster(HttpSession s) {
14           if(!rol(s,"Yönetici")) return "";
15           return form(s,"/mesai","Çalışma
15          aralığı",k.personel()+alan("gun","Gün","date")+alan("ilk","Başlangıç","time")+alan("son","Bitiş","time"),"Ara
15          lık ekle");
16       }
17
18       @PostMapping("/mesai")
19       String mesai(HttpSession s,@RequestParam long kisi,@RequestParam LocalDate gun,@RequestParam LocalTime
19          ilk,@RequestParam LocalTime son) {
20           yetki(s,"Yönetici");
21           if(gun.isBefore(LocalDate.now())) throw new IllegalArgumentException("Geçmiş gün seçilemez.");
22           int n=k.db().update("insert into mesai select p.id,?,?,? from kisi p join bolum b on b.id=p.bolum join kurum
22           k on k.id=b.kurum where p.id=? and k.ac<=? and k.kapa>=?",gun,ilk,son,kisi,ilk,son);
23           if(n!=1) throw new IllegalArgumentException("Aralık şubenin çalışma saatleri içinde olmalı.");
24           return "redirect:/";
25       }
26   }
```

Takvim / 1–32

Beş dakika aralıkla üretilen aday başlangıçlar incelenir. Muayenenin tamamı mesaiye sığmalı, hem hasta hem çalışan için boş olmalıdır.

```
1    package tr.ndp.hastane;
2
3    import java.time.*;
4    import java.util.*;
5    import org.springframework.jdbc.core.JdbcTemplate;
6    import org.springframework.stereotype.Service;
7
8    @Service
9    public record Takvim(JdbcTemplate db) {
10       public record Secim(long personel, long hizmet, LocalDate gun) {}
11       record Aralik(LocalTime ilk, LocalTime son) {}
12       public record Teklif(SortedSet<LocalTime> saatler, int sure, java.math.BigDecimal ucret) {}
13
14       public Teklif hesapla(long hasta, Secim secim) {
15           var h = db.queryForMap("""
16                   select h.sure,h.ucret from hizmet h join sunum s on h.id=s.hizmet
17                   where s.kisi=? and s.hizmet=?
18                   """, secim.personel(), secim.hizmet());
19           int sure = ((Number)h.get("sure")).intValue();
20           var dolu = db.query("""
21                   select ilk,son from randevu where gun=? and (hasta=? or personel=?)
22                   and durum in ('Bekliyor','Onaylandı')
23                   """, (rs,i) -> new Aralik(rs.getTime(1).toLocalTime(), rs.getTime(2).toLocalTime()),
24                   secim.gun(), hasta, secim.personel());
25           var mesai = db.query("select ilk,son from mesai where gun=? and kisi=?",
26                   (rs,i) -> new Aralik(rs.getTime(1).toLocalTime(), rs.getTime(2).toLocalTime()),
27                   secim.gun(),secim.personel());
28           SortedSet<LocalTime> saatler = new TreeSet<>();
29           var simdi = LocalDateTime.now();
30           for (Aralik m : mesai) {
31               for (int dk=m.ilk().toSecondOfDay()/60; dk+sure<=m.son().toSecondOfDay()/60; dk+=5) {
32                   var ilk = LocalTime.ofSecondOfDay(dk*60L);
```

Takvim / 33–40

Beş dakika aralıkla üretilen aday başlangıçlar incelenir. Muayenenin tamamı mesaiye sığmalı, hem hasta hem çalışan için boş olmalıdır.

```
33                  var son = ilk.plusMinutes(sure);
34                  boolean bos = dolu.stream().noneMatch(a -> ilk.isBefore(a.son()) && son.isAfter(a.ilk()));
35                  if (bos && secim.gun().atTime(ilk).isAfter(simdi)) saatler.add(ilk);
36              }
37          }
38          return new Teklif(saatler,sure,(java.math.BigDecimal)h.get("ucret"));
39      }
40  }
```

SaatSecimi / 1–29

GET /saatler seçilen çalışan, işlem ve gün için boş saatleri getirir. Kullanıcı uygun başlangıcı rezervasyon formuyla gönderir.

```
1    package tr.ndp.hastane;
2
3    import jakarta.servlet.http.HttpSession;
4    import java.time.*;
5    import org.springframework.jdbc.core.JdbcTemplate;
6    import org.springframework.core.annotation.Order;
7    import org.springframework.stereotype.Controller;
8    import org.springframework.web.bind.annotation.*;
9    import static tr.ndp.hastane.Sayfa.*;
10
11   @Controller
12   @Order(8)
13   public record SaatSecimi(JdbcTemplate db,Takvim takvim,java.util.Optional<Rezervasyon> rezervasyon) implements Bolme
13           {
14       public String goster(HttpSession s) {
15           if(!rol(s,"Hasta")) return "";
16           var liste=db.queryForList("""
17                   select concat(p.id, '-', h.id) id, concat(k.ad,' / ',b.ad,' / ',p.ad,' (',p.uzmanlik,') / ',h.ad,' ·
17           ',h.sure,' dk · ',h.ucret,' TL') ad
18                   from sunum x join kisi p on p.id=x.kisi join hizmet h on h.id=x.hizmet
19                   join bolum b on b.id=h.bolum join kurum k on k.id=b.kurum order by k.ad,b.ad,p.ad
20                   """);
21           return "<section><h2>Randevu ara</h2><form action='/saatler'>"+sec("secim","Çalışan ve
21          hizmet",liste)+alan("gun","Gün","date")+"<button>Uygun saatleri bul</button></form></section>";
22       }
23
24       @GetMapping(value="/saatler",produces="text/html;charset=UTF-8")
25       @ResponseBody
26       String saatler(HttpSession s,@RequestParam String secim,@RequestParam LocalDate gun) {
27           long hasta=yetki(s,"Hasta");
28           if(!secim.matches("[0-9]+-[0-9]+")) throw new IllegalArgumentException("Hizmet seçimi geçersiz.");
29           String[] ikili=secim.split("-");
```

SaatSecimi / 30–41

GET /saatler seçilen çalışan, işlem ve gün için boş saatleri getirir. Kullanıcı uygun başlangıcı rezervasyon formuyla gönderir.

```
30          var teklif=takvim.hesapla(hasta,new Takvim.Secim(Long.parseLong(ikili[0]),Long.parseLong(ikili[1]),gun));
31          StringBuilder html=new StringBuilder("<section><h2>"+gun+" · Uygun saatler</h2>");
32          if(teklif.saatler().isEmpty()) html.append("<p>Bu gün için boş saat yok.</p>");
33          else if(rezervasyon.isEmpty()) html.append(teklif.saatler());
34          else {
35              html.append("<form method='post' action='/ayir'><input type='hidden' name='anahtar'
35          value='"+s.getAttribute("anahtar")+"'><input type='hidden' name='kisi' value='"+ikili[0]+"'><input
35          type='hidden' name='hizmet' value='"+ikili[1]+"'><input type='hidden' name='gun'
35          value='"+gun+"'><label>Başlangıç<select name='ilk'>");
36              teklif.saatler().forEach(t -> html.append("<option>").append(t).append("</option>"));
37              html.append("</select></label><button>Randevu talebi oluştur</button></form>");
38          }
39          return cerceve(html+"</section><a href='/'>Panele dön</a>");
40      }
41  }
```

src/main/java/tr/ndp/hastane/Rezervasyon.java

Rezervasyon / 1–24

Ortak kilit alındıktan sonra takvim yeniden hesaplanır. Böylece iki tarayıcı aynı saati görmüş olsa bile yalnız bir talep yazılır.

```
1    package tr.ndp.hastane;
2
3    import java.time.LocalTime;
4    import org.springframework.stereotype.Service;
5    import org.springframework.transaction.annotation.Transactional;
6
7    @Service
8    public class Rezervasyon {
9        private final Takvim takvim;
10
11       public Rezervasyon(Takvim takvim) { this.takvim = takvim; }
12
13       @Transactional
14       public void ayir(long hasta, Takvim.Secim secim, LocalTime ilk) {
15           var db = takvim.db();
16           db.queryForObject("select id from kilit where id=1 for update", Integer.class);
17           var teklif = takvim.hesapla(hasta, secim);
18           if (!teklif.saatler().contains(ilk)) throw new IllegalArgumentException("Bu saat uygun değil.");
19           db.update("""
20                   insert into randevu(hasta,personel,hizmet,gun,ilk,son,ucret,durum)
21                   values(?,?,?,?,?,?,?,'Bekliyor')
22                   """,
22              hasta,secim.personel(),secim.hizmet(),secim.gun(),ilk,ilk.plusMinutes(teklif.sure()),teklif.ucret());
23       }
24   }
```

Randevular / 1–30

POST /ayir talebi kaydeder. Listelemede hasta ve çalışan kendi kayıtlarını, yönetici tüm şubeleri görür.

```
1    package tr.ndp.hastane;
2
3    import java.time.*;
4    import jakarta.servlet.http.HttpSession;
5    import org.springframework.core.annotation.Order;
6    import org.springframework.jdbc.core.JdbcTemplate;
7    import org.springframework.stereotype.Controller;
8    import org.springframework.web.bind.annotation.*;
9    import static tr.ndp.hastane.Sayfa.*;
10
11   @Controller
12   @Order(9)
13   public record Randevular(JdbcTemplate db,Rezervasyon servis) implements Bolme {
14       public String goster(HttpSession s) {
15           if(s.getAttribute("id")==null) return "";
16           String sql="""
17                   select r.id,k.ad şube,b.ad bölüm,h.ad hizmet,a.ad hasta,p.ad çalışan,r.gun gün,
18                   r.ilk başlangıç,r.son bitiş,datediff('minute',r.ilk,r.son) dakika,r.ucret ücret,r.durum
19                   from randevu r join kisi a on a.id=r.hasta join kisi p on p.id=r.personel
20                   join hizmet h on h.id=r.hizmet join bolum b on b.id=h.bolum join kurum k on k.id=b.kurum
21                   """;
22           String sira=" order by r.gun,r.ilk";
23           var liste=rol(s,"Yönetici") ? db.queryForList(sql+sira) :
24                   db.queryForList(sql+(rol(s,"Hasta") ? " where r.hasta=?" : " where
24              r.personel=?")+sira,s.getAttribute("id"));
25           return "<section><h2>Randevular</h2>"+tablo(liste)+"</section>";
26       }
27
28       @PostMapping("/ayir")
29       String ayir(HttpSession s,@RequestParam long kisi,@RequestParam long hizmet,@RequestParam LocalDate
29          gun,@RequestParam LocalTime ilk) {
30           servis.ayir(yetki(s,"Hasta"),new Takvim.Secim(kisi,hizmet,gun),ilk);
```

Randevular / 31–33

POST /ayir talebi kaydeder. Listelemede hasta ve çalışan kendi kayıtlarını, yönetici tüm şubeleri görür.

```
31            return "redirect:/";
32        }
33    }
```

Kararlar / 1–27

Yalnızca seçilen çalışanın bekleyen talebi güncellenir. Onay saati kapalı tutar. Ret sonrası saat diğer hastalara açılır.

```
1    package tr.ndp.hastane;
2
3    import jakarta.servlet.http.HttpSession;
4    import org.springframework.core.annotation.Order;
5    import org.springframework.jdbc.core.JdbcTemplate;
6    import org.springframework.stereotype.Controller;
7    import org.springframework.web.bind.annotation.*;
8    import static tr.ndp.hastane.Sayfa.*;
9
10   @Controller
11   @Order(10)
12   public record Kararlar(JdbcTemplate db) implements Bolme {
13       public String goster(HttpSession s) {
14           if(!rol(s,"Doktor") && !rol(s,"Sağlık personeli")) return "";
15           var liste=db.queryForList("select r.id,concat(a.ad,' · ',h.ad,' · ',r.gun,' ',r.ilk) ad from randevu r join
15          kisi a on a.id=r.hasta join hizmet h on h.id=r.hizmet where r.personel=? and
15          r.durum='Bekliyor'",s.getAttribute("id"));
16           if(liste.isEmpty()) return "<section><h2>Bekleyen talepler</h2><p>Değerlendirilecek talep
16          yok.</p></section>";
17           return form(s,"/karar","Talep değerlendir",sec("id","Randevu",liste)+"<label>Karar<select
17          name='karar'><option>Onaylandı</option><option>Reddedildi</option></select></label>","Kararı kaydet");
18       }
19
20       @PostMapping("/karar")
21       String karar(HttpSession s,@RequestParam long id,@RequestParam String karar) {
22           long kisi=yetki(s,"Doktor","Sağlık personeli");
23           if(!karar.equals("Onaylandı") && !karar.equals("Reddedildi")) throw new IllegalArgumentException("Geçersiz
23          karar.");
24           if(db.update("update randevu set durum=? where id=? and personel=? and durum='Bekliyor'",karar,id,kisi)!=1)
25               throw new IllegalArgumentException("Bu talep artık değerlendirilemiyor.");
26           return "redirect:/";
27       }
```

Kararlar / 28–28

Yalnızca seçilen çalışanın bekleyen talebi güncellenir. Onay saati kapalı tutar. Ret sonrası saat diğer hastalara açılır.

```
28    }
```

WebAkisiTest / 1–32

Ayrı web oturumları kullanılarak şube tanımından hastanın onay sonucunu görmesine kadar form istekleri sınanır.

```
1    package tr.ndp.hastane;
2
3    import java.time.LocalDate;
4    import org.junit.jupiter.api.Test;
5    import org.springframework.beans.factory.annotation.Autowired;
6    import org.springframework.boot.test.autoconfigure.web.servlet.AutoConfigureMockMvc;
7    import org.springframework.boot.test.context.SpringBootTest;
8    import org.springframework.mock.web.MockHttpSession;
9    import org.springframework.test.web.servlet.MockMvc;
10   import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.*;
11   import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.*;
12   import static org.hamcrest.Matchers.containsString;
13
14   @SpringBootTest(properties="spring.datasource.url=jdbc:h2:mem:webakisi")
15   @AutoConfigureMockMvc
16   class WebAkisiTest {
17       @Autowired MockMvc mvc;
18
19       void yaz(MockHttpSession s,String yol,String... veri) throws Exception {
20           var istek=post(yol).session(s).param("anahtar",(String)s.getAttribute("anahtar"));
21           for(int i=0;i<veri.length;i+=2) istek.param(veri[i],veri[i+1]);
22           mvc.perform(istek).andExpect(status().is3xxRedirection());
23       }
24
25       MockHttpSession oturum() throws Exception {
26           var s=new MockHttpSession();
27           mvc.perform(get("/").session(s)).andExpect(status().isOk());
28           return s;
29       }
30
31       @Test
32       void ilkKurulumdanOnaya() throws Exception {
```

WebAkisiTest / 33–58

Ayrı web oturumları kullanılarak şube tanımından hastanın onay sonucunu görmesine kadar form istekleri sınanır.

```
33          var y=oturum();
34          mvc.perform(post("/kurulum").session(y)).andExpect(status().isForbidden());
35          yaz(y,"/kurulum","ad","Yönetici","hesap","yon","parola","deneme123");
36          yaz(y,"/giris","hesap","yon","parola","deneme123");
37          yaz(y,"/kurum","ad","Merkez","ac","09:00","kapa","17:00");
38          yaz(y,"/bolum","kurum","1","ad","Dahiliye");
39          yaz(y,"/hizmet","bolum","1","ad","Muayene","sure","30","ucret","100.50");
40
40         yaz(y,"/calisan","ad","Doktor","hesap","dr","parola","deneme123","bolum","1","uzmanlik","Dahiliye","tur","Dok
40         tor");
41          yaz(y,"/ata","kisi","2","hizmet","1");
42          String gun=LocalDate.now().plusDays(1).toString();
43          yaz(y,"/mesai","kisi","2","gun",gun,"ilk","09:00","son","10:00");
44          var h=oturum();
45          yaz(h,"/hasta","ad","Hasta","hesap","hasta","parola","deneme123");
46          yaz(h,"/giris","hesap","hasta","parola","deneme123");
47          mvc.perform(get("/").session(h)).andExpect(content().string(containsString("Merkez / Dahiliye")));
48          mvc.perform(get("/saatler").session(h).param("secim","2-1").param("gun",gun))
49                  .andExpect(content().string(containsString("<option>09:00</option>")));
50          yaz(h,"/ayir","kisi","2","hizmet","1","gun",gun,"ilk","09:00");
51          mvc.perform(post("/karar").session(h).param("anahtar",(String)h.getAttribute("anahtar")).param("id","1").par
51         am("karar","Onaylandı")).andExpect(status().isBadRequest());
52          var dr=oturum();
53          yaz(dr,"/giris","hesap","dr","parola","deneme123");
54          mvc.perform(get("/").session(dr)).andExpect(content().string(containsString("Talep değerlendir")));
55          yaz(dr,"/karar","id","1","karar","Onaylandı");
56          mvc.perform(get("/").session(h)).andExpect(content().string(containsString("Onaylandı")));
57      }
58  }
```

RandevuTest / 1–32

Aynı saate iki eşzamanlı talep gönderilir. Yalnız bir kaydın oluşması, bitişik saatin ve ret sonrası ilk saatin uygunluğu denetlenir.

```
1    package tr.ndp.hastane;
2
3    import java.time.*;
4    import java.util.concurrent.*;
5    import org.junit.jupiter.api.Test;
6    import org.springframework.beans.factory.annotation.Autowired;
7    import org.springframework.boot.test.context.SpringBootTest;
8    import org.springframework.jdbc.core.JdbcTemplate;
9    import static org.junit.jupiter.api.Assertions.*;
10
11   @SpringBootTest(properties={"hastane.menu=false","spring.datasource.url=jdbc:h2:mem:sinav"})
12   class RandevuTest {
13       @Autowired JdbcTemplate db;
14       @Autowired Rezervasyon servis;
15       @Autowired Takvim takvim;
16
17       @Test
18       void eszamanliTaleplerVeSinirSaatleri() throws Exception {
19           db.update("insert into kurum(ad,ac,kapa) values('Merkez','09:00','17:00')");
20           db.update("insert into bolum(kurum,ad) values(1,'Dahiliye')");
21           db.update("insert into hizmet(bolum,ad,sure,ucret) values(1,'Muayene',30,125.50)");
22           db.update("insert into kisi(hesap,ad,parola,rol,bolum) values('d','Doktor','x','Doktor',1)");
23           db.update("insert into kisi(hesap,ad,parola,rol) values('a','Ali','x','Hasta'),('b','Ece','x','Hasta')");
24           db.update("insert into sunum values(1,1)");
25           LocalDate gun = LocalDate.now().plusDays(1);
26           db.update("insert into mesai values(1,?,'09:00','10:00')",gun);
27           var secim = new Takvim.Secim(1,1,gun);
28           var ilk = LocalTime.of(9,0);
29           var hazir = new CyclicBarrier(2);
30           var havuz = Executors.newFixedThreadPool(2);
31           try {
32               Callable<Boolean> talep = () -> {
```

RandevuTest / 33–52

Aynı saate iki eşzamanlı talep gönderilir. Yalnız bir kaydın oluşması, bitişik saatin ve ret sonrası ilk saatin uygunluğu denetlenir.

```
33                  hazir.await(5,TimeUnit.SECONDS);
34                  try {
35                      servis.ayir(2,secim,ilk);
36                      return true;
37                  } catch (IllegalArgumentException e) {
38                      return false;
39                  }
40              };
41              var a = havuz.submit(talep);
42              var b = havuz.submit(talep);
43              assertNotEquals(a.get(10,TimeUnit.SECONDS),b.get(10,TimeUnit.SECONDS));
44          } finally {
45              havuz.shutdownNow();
46          }
47          assertEquals(1,db.queryForObject("select count(*) from randevu",Integer.class));
48          assertEquals(java.util.Set.of(ilk.plusMinutes(30)),takvim.hesapla(3,secim).saatler());
49          db.update("update randevu set durum='Reddedildi'");
50          assertTrue(takvim.hesapla(3,secim).saatler().contains(ilk));
51      }
52  }
```